

## Code-Layout, Readability and Reusability

|               |                                                      |
|---------------|------------------------------------------------------|
| Date          | 5 July 2026                                          |
| Team ID       |                                                      |
| Project Name  | AI Powered Debt Relief & Financial Recovery Platform |
| Maximum Marks | 5 Marks                                              |

### Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

### Code Layout Checklist:

| S.No | Code Quality Parameter    | Description                                     | Followed (Yes/No/Partial) | Remarks                                                                             |
|------|---------------------------|-------------------------------------------------|---------------------------|-------------------------------------------------------------------------------------|
| 1    | Consistent Indentation    | Uniform spacing/tabs used throughout the code   | Yes                       | 4-space indentation followed in all Python files (PEP 8)                            |
| 2    | Proper File Structure     | Files and folders are logically organized       | Yes                       | Backend split into routers/, services/, models.py, schemas.py, auth.py, database.py |
| 3    | Meaningful Variable Names | Variables reflect their purpose clearly         | Yes                       | e.g. financial_health_score, existing_debts, monthly_income                         |
| 4    | Function / Method Names   | Functions are descriptively named               | Yes                       | snake_case naming, e.g. calculate_dti_ratio(), predict_settlement()                 |
| 5    | Code Comments             | Inline and block comments explain logic clearly | Yes                       | Docstrings included on key modules (e.g. database.py, ai_engine.py)                 |
| 6    | Modular Design            | Code is split into reusable functions/modules   | Yes                       | Separate services for financial_engine, settlement, and ai_engine                   |
| 7    | No Redundant Code         | Duplicate or unused code is removed             | Yes                       | __pycache__ and .pytest_cache cleaned before final submission                       |
| 8    | Error Handling            | Exceptions and errors are handled gracefully    | Yes                       | Try/except used around Gemini API calls with automatic rule-based fallback          |

### Reusable Components / Modules:

| S.No | Component / Module Name                | Language / Technology                 | Where Reused                                    | Reusability Level (High/Medium/Low) |
|------|----------------------------------------|---------------------------------------|-------------------------------------------------|-------------------------------------|
| 1    | Authentication Module (auth.py)        | Python (FastAPI, python-jose, bcrypt) | User Registration, Login, Token Validation      | High                                |
| 2    | Financial Engine (financial_engine.py) | Python                                | Dashboard, Financial Health page, Debt Timeline | High                                |

| S.No | Component / Module Name            | Language / Technology         | Where Reused                                           | Reusability Level (High/Medium/Low) |
|------|------------------------------------|-------------------------------|--------------------------------------------------------|-------------------------------------|
| 3    | Settlement Service (settlement.py) | Python                        | Settlement Predictor page                              | High                                |
| 4    | AI Engine (ai_engine.py)           | Python (Google Generative AI) | AI Negotiation Strategy, Settlement Letter Generation  | High                                |
| 5    | Database Connector (database.py)   | Python (SQLAlchemy)           | All database operations across every router            | High                                |
| 6    | API Client (client.js)             | JavaScript (Axios)            | All frontend-to-backend requests, JWT header injection | High                                |

**Overall Code Quality Assessment:**

| Aspect                   | Rating (1-5) | Comments                                                                                                           |
|--------------------------|--------------|--------------------------------------------------------------------------------------------------------------------|
| Code Layout & Structure  | 5            | Well-structured, modular codebase with clean separation of routers, services, and models.                          |
| Readability              | 5            | Code is easy to read and understand — descriptive names, consistent style, and clear logic.                        |
| Reusability              | 5            | Financial engine, settlement, and AI modules are cleanly reusable and independent of route handlers.               |
| Documentation / Comments | 4            | Key modules are documented with docstrings; some complex calculation logic could use more inline comments.         |
| Overall Score            | 19 / 20      | Excellent code quality, backed by 86 passing backend tests. Maintain and continue following good coding practices. |